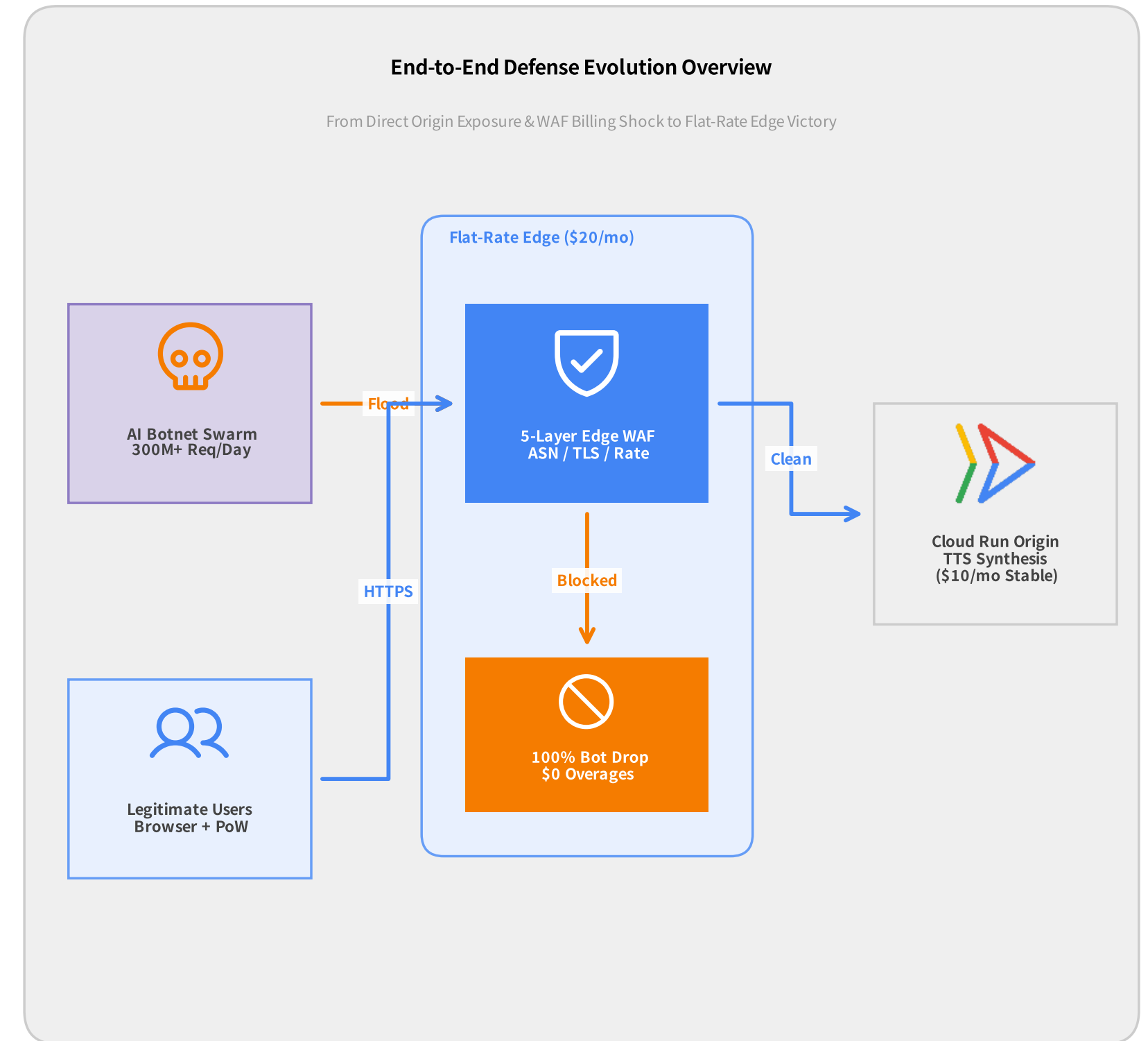


Surviving a 300M+ Req/Day DDoS & EDoS Attack

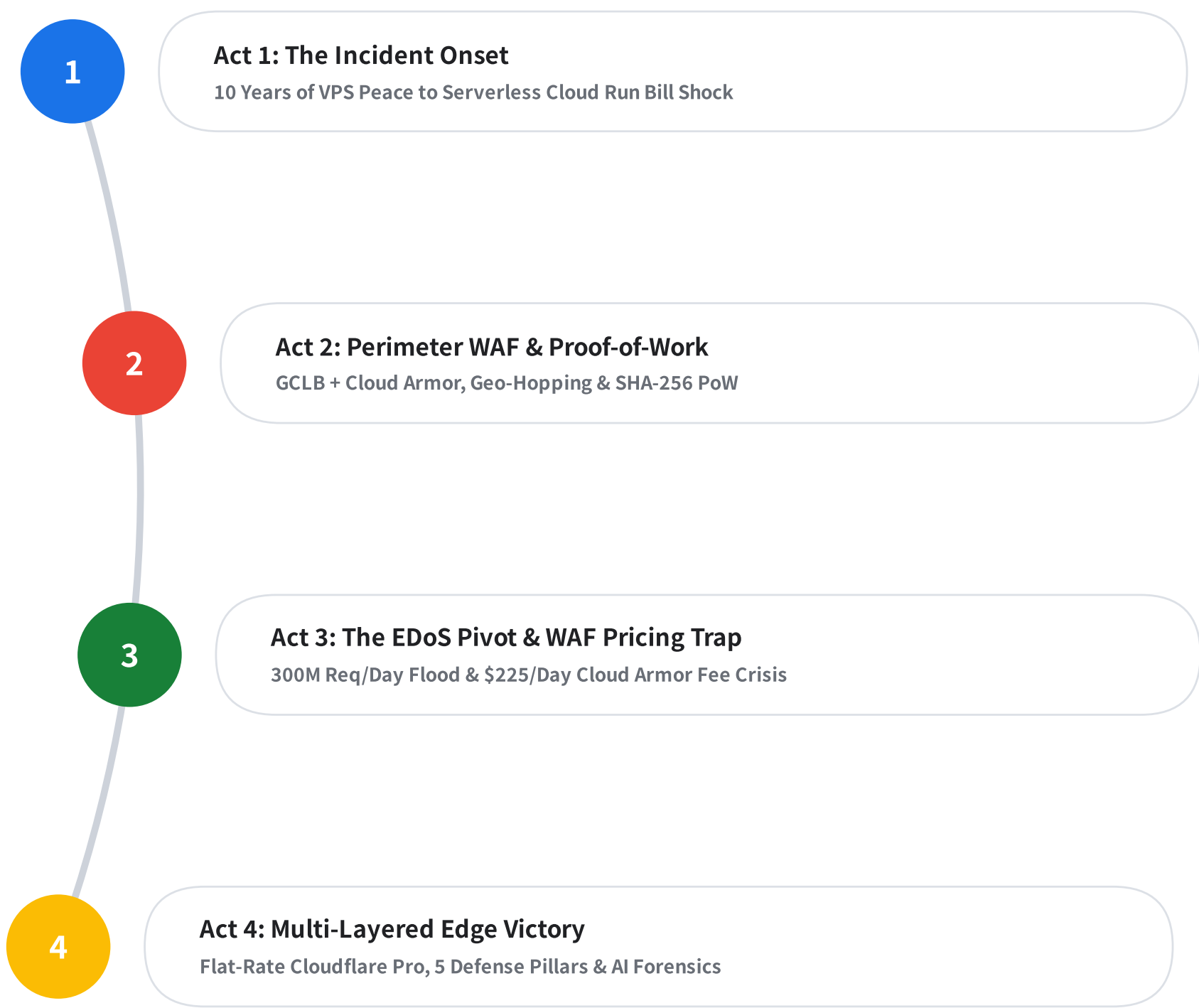
A Real-World Serverless Postmortem on Google Cloud Run & Edge WAF Architecture

How an AI-orchestrated botnet pushed a 10-year hobby speech synthesis service from \$10/month to a \$6,750/month WAF billing crisis — and how 5-layer edge defense defeated it for \$20/month flat.

Incident Postmortem & Cloud Security Case Study · 2026



Agenda & Incident Executive Summary



300M+	Peak Attack Volume / Day Volumetric HTTP flood across 5,000+ nodes
\$6,750	Projected Monthly WAF Bill \$225/day Cloud Armor rule evaluation trap
<50ms	Browser SHA-256 PoW Asymmetric Web Worker hash puzzle
\$20/mo	Final Flat-Rate Defense Cloudflare Pro + 5 orthogonal edge layers

ACT / CHAPTER

01



Act 1 of 4

The Incident Onset

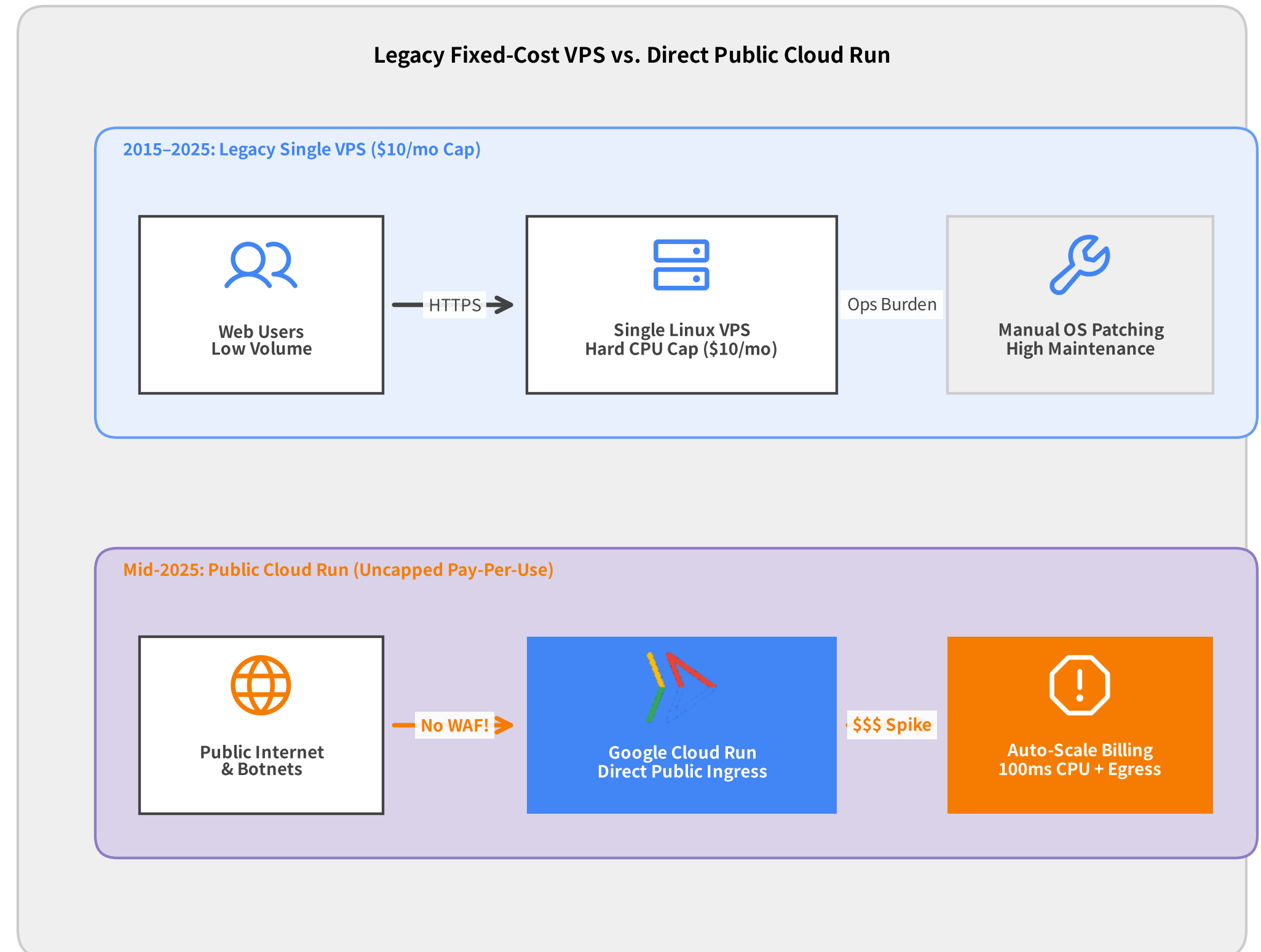
From 10 Years of VPS Peace to Serverless Cloud Run Bill Shock

- 1.1 10-Year Legacy VPS vs. Public Serverless Cloud Run Exposure
- 1.2 Cache-Busting TTS Synthesis Flood & 4-5x Weekly Bill Spike
- 1.3 Initial Triage: Why Origin Clamping Fails vs. Edge Drop

Background: A 10-Year Hobby Service Enters the Cloud

Architecture Evolution

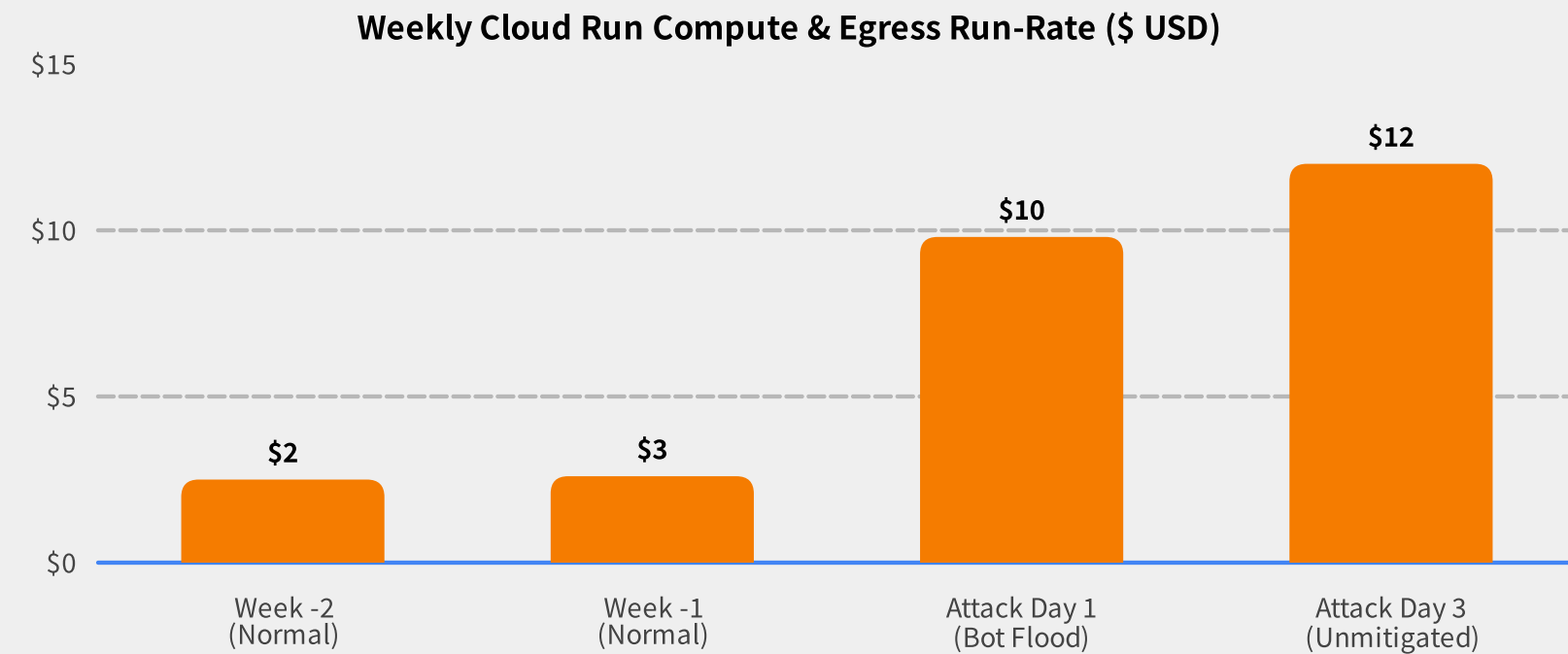
- **10 Years of Peace (2015–2025)** Japanese TTS web service running at **\$10/mo** on a single VPS.
- **Serverless Migration (Mid-2025)** Containerized onto **Google Cloud Run** for zero-ops maintenance.
- **Hidden Vulnerability** Direct public URL (`run.app`) with **no CDN or Edge WAF** and uncapped pay-per-use scaling.



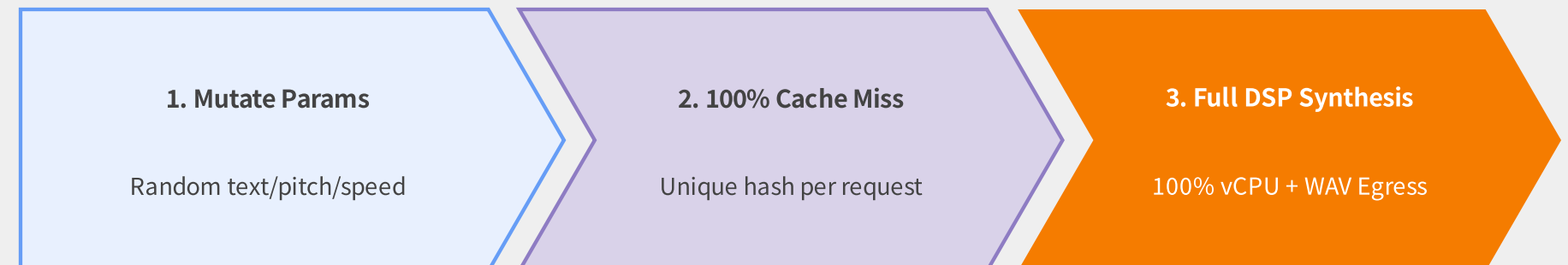
The Attack Onset: Cache-Busting TTS Flood

Telemetry & Billing Shock

- **4–5x Weekly Bill Multiplier** Cloud Run compute & egress costs jumped **400%+** overnight.
- **100% Cache-Miss Exploitation** Bots randomized `text`, `pitch`, and `speed` query parameters on every call.
- **CPU Saturation** Every single request forced a cold DSP audio synthesis cycle.

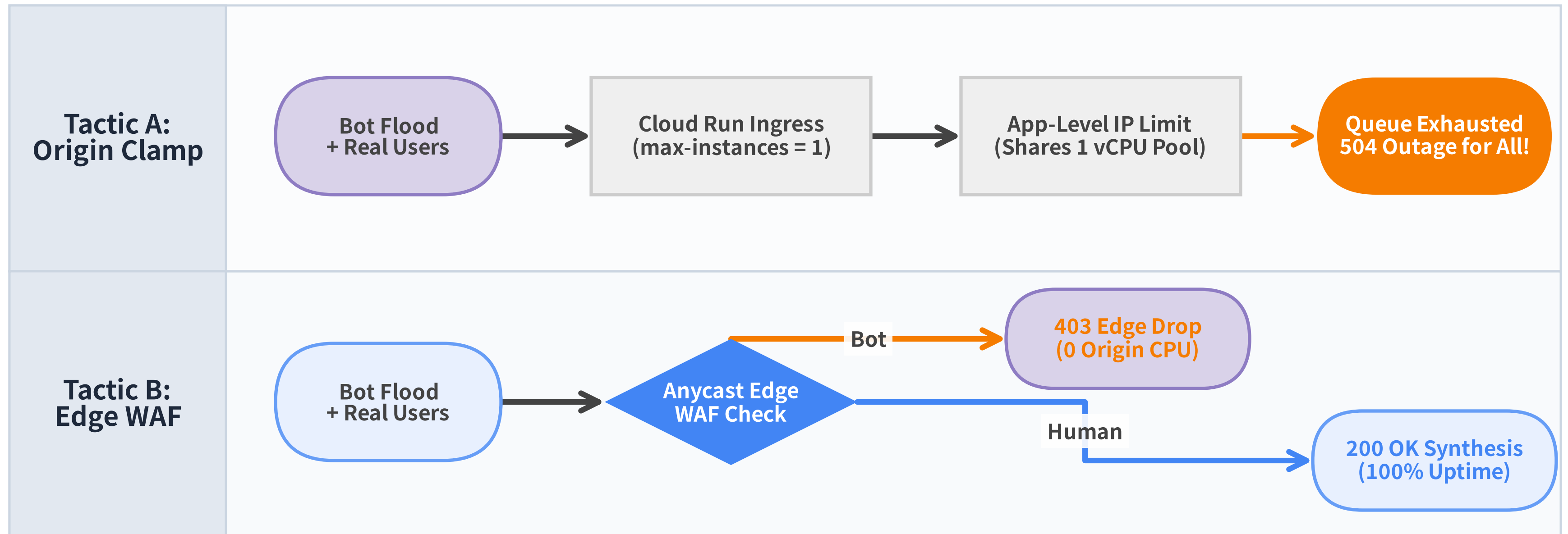


How Parameter Mutation Defeated Application Caching



Initial Triage: Origin Clamping vs. Edge Filtering

- **Tactic A (`max-instances=1` + App Rate Limit)**: Caps runaway Cloud bill immediately, but **starves legitimate users (504 Gateway Timeout)** as bots fill the queue.
- **Tactic B (Dedicated Edge WAF Layer)**: Drops malicious floods at the **Anycast POP (403 Forbidden)** so only clean traffic reaches the Cloud Run container.



ACT / CHAPTER

02



Act 2 of 4

Perimeter WAF & Proof-of-Work

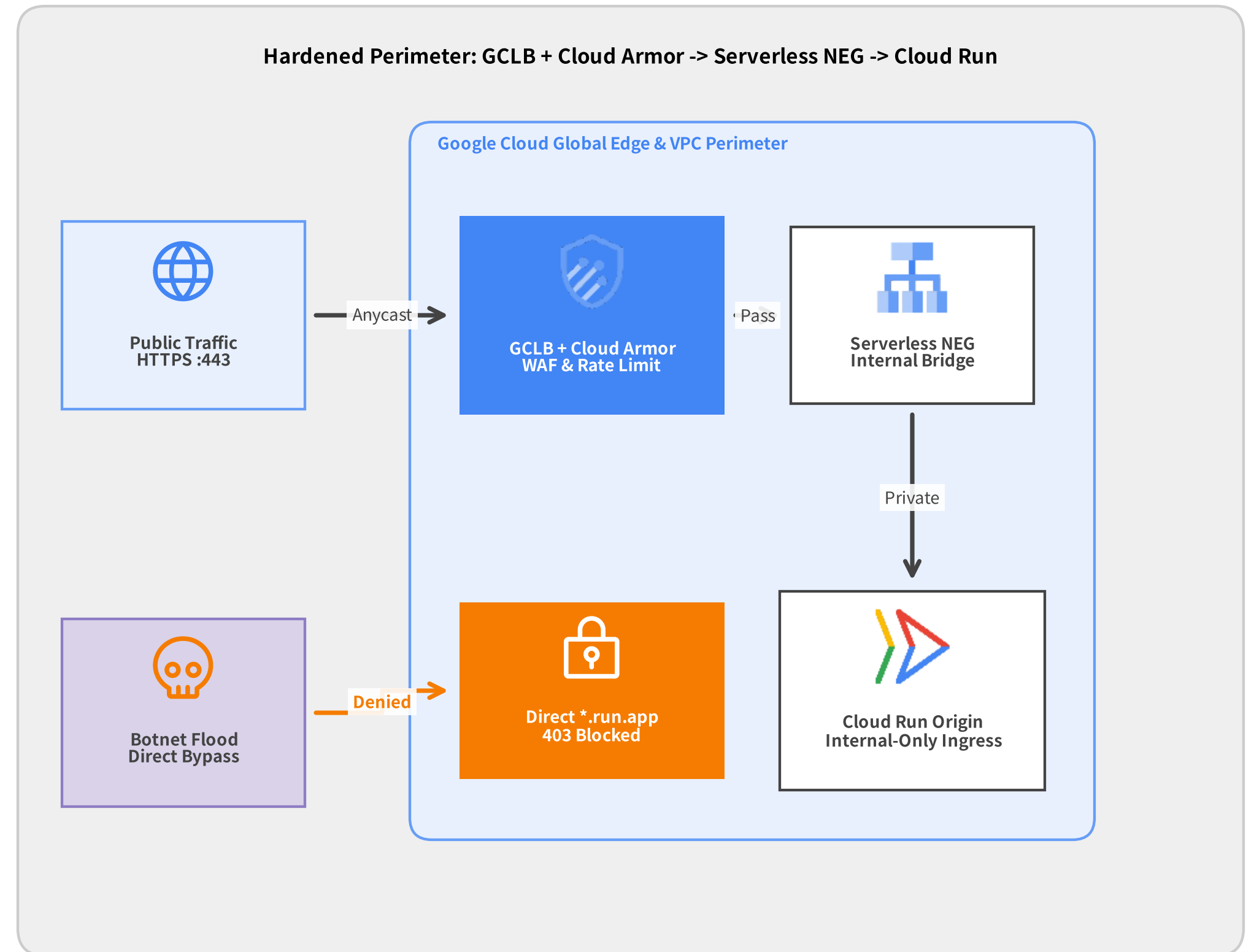
Cat-and-Mouse Geo-Blocking, JS Header Spoofing, and Asymmetric PoW

- 2.1 Perimeter Hardening with GCLB Anycast Edge & Cloud Armor WAF
- 2.2 Why Geo-Blocking Failed: 4 Rapid Proxy Hopping Waves
- 2.3 Static JS Token Spoofing vs. Asymmetric SHA-256 Proof-of-Work

Perimeter Defense: GCLB & Cloud Armor WAF

Edge Shield Architecture

- **Private Origin Isolation** Locked down `run.app` direct URL (`ingress = internal-and-cloud-load-balancing`).
- **Global Anycast Ingress** Placed **Google Cloud Load Balancing (GCLB)** with **Serverless NEG** in front of Cloud Run.
- **Edge L7 WAF Policies** Attached **Cloud Armor** for edge IP rate limiting and geo-filtering before container invocation.



Why Geo-Blocking Failed: The Proxy Whack-a-Mole

- **Initial Hypothesis:** Since our TTS service targets Japanese users, blocking overseas attack origins in Cloud Armor should stop the flood.
- **Reality:** Within **24 hours**, the botnet hopped across **4 proxy waves**, ending inside **Japanese domestic residential ISPs** where geo-blocking is useless.

24-Hour Geographic Proxy Escalation Timeline

Wave 1: Single Region

95% China (CN) IPs



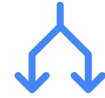
WAF Rule #1

Block CN Region

Bypassed in 15 mins

Wave 2: Regional Shift

SG / VN / ID / BR / US



WAF Rule #2

Block 10+ Countries

Bypassed in 1 hour

Wave 3: Global Mesh

120+ Countries (JP-Only Rule)



WAF Rule #3

Allow ONLY Japan (JP)

Bypassed in 4 hours

Wave 4: Domestic JP

JP Residential ISPs + Cloud



Geo-Block Defeated

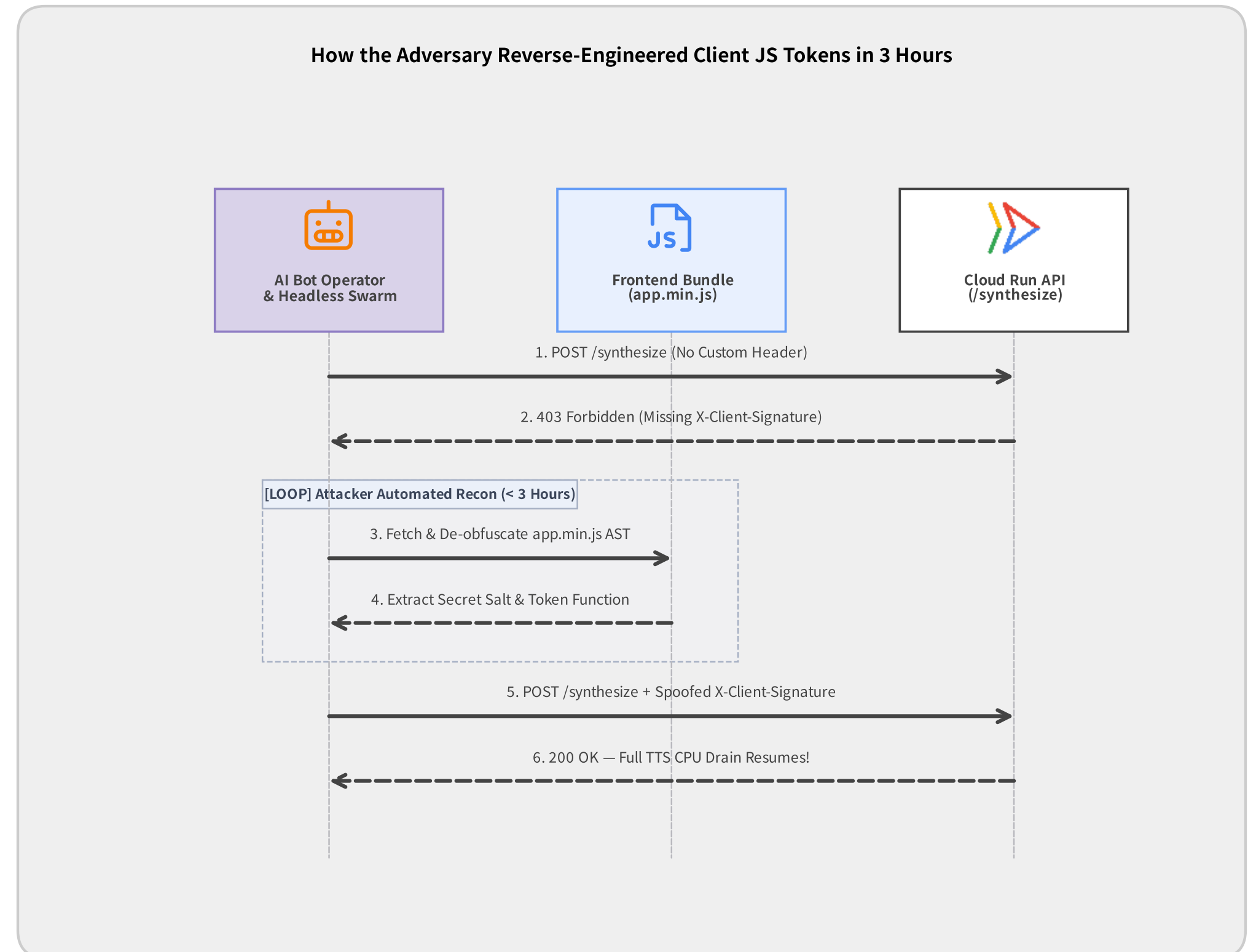
Attacker inside JP ISPs!

Cannot block own users

Application-Layer Tactics: Static JS Token Spoofing

Why "Secret" JS Logic Failed

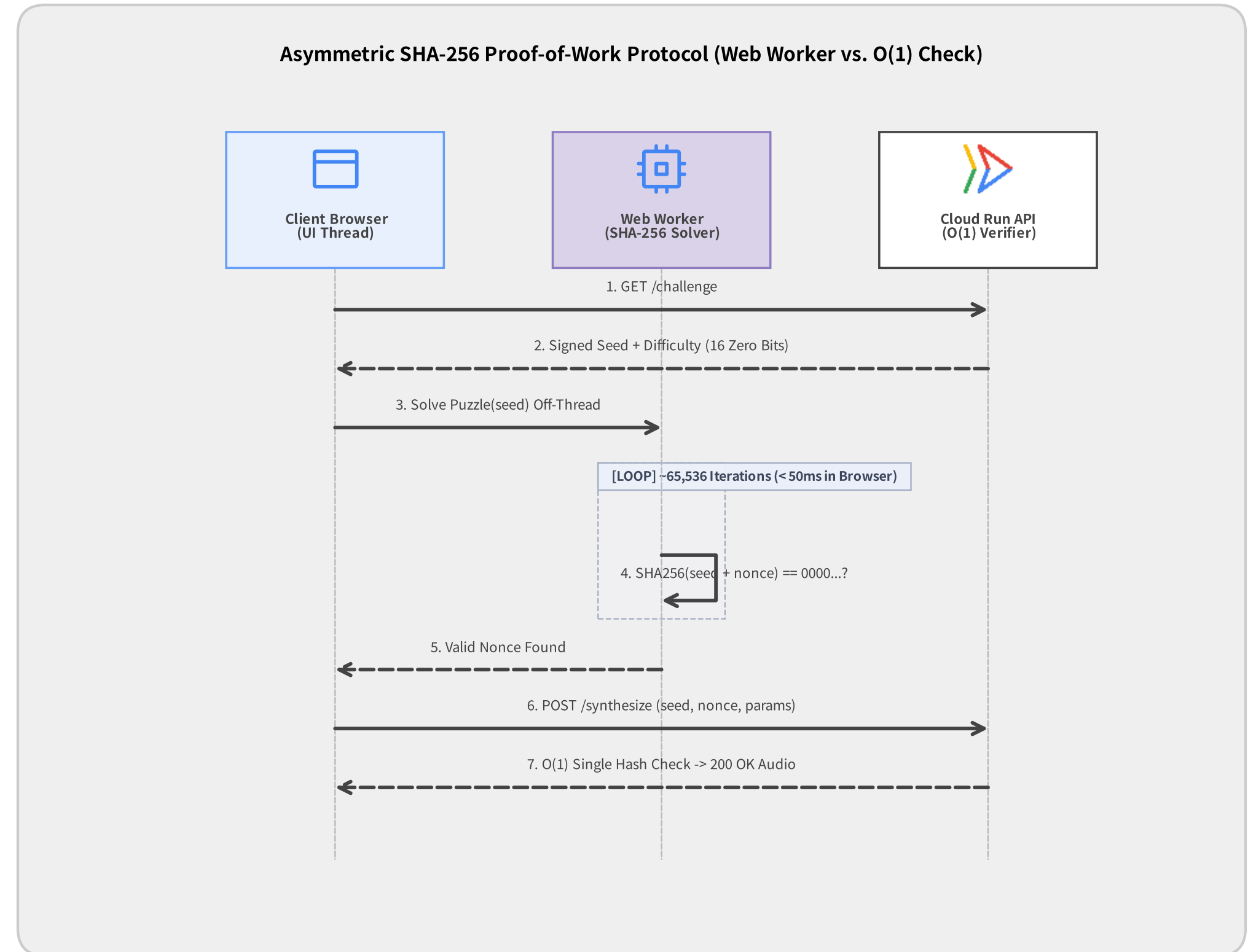
- **Custom Header Verification** Added obfuscated frontend JS to compute `X-Client-Signature` on `/synthesize` calls.
- **3 Hours of Relief** Simple HTTP flood bots were dropped immediately (`403 Forbidden`).
- **Automated AST Reverse-Engineering** Within **3 hours**, the attacker parsed our JS bundle and replicated the exact HMAC algorithm in their headless bot scripts.



Asymmetric Defense: SHA-256 Proof-of-Work

Cryptographic Cost Inversion

- **One-Time Challenge Seed** Server issues an HMAC-signed `seed` with a short TTL via `GET /challenge`.
- **Browser Web Worker (<50ms)** Client finds a `nonce` where `SHA256(seed + nonce)` starts with `0000...` (~65k hashes).
- **100% Synthesis Protection** Server verifies in **O(1)** (`1 hash`), while 1,000 RPS costs the botnet **65M+ hashes/sec**.





The EDoS Pivot & WAF Pricing Trap

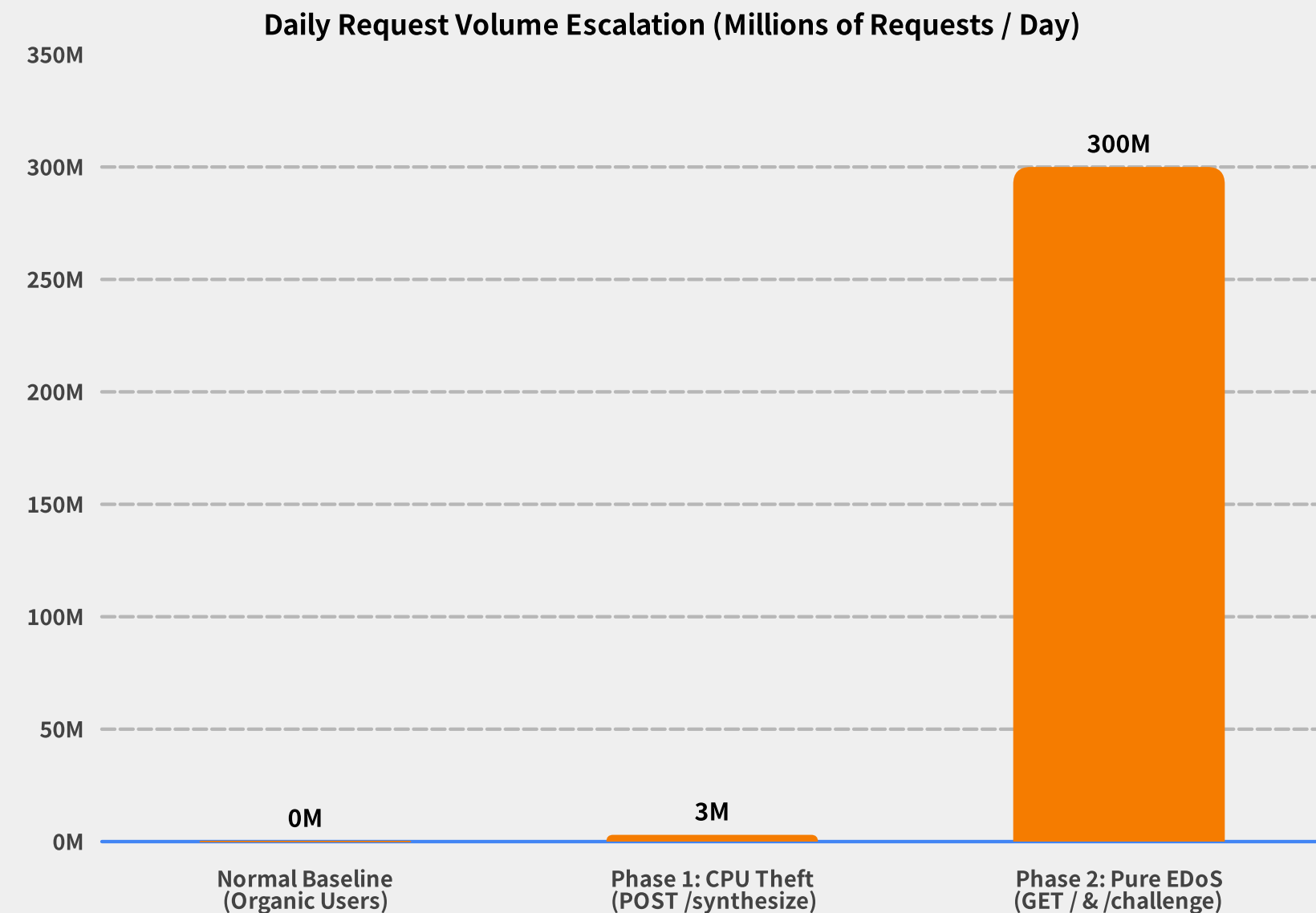
When a 99.9% Effective WAF Threatens to Bankrupt the Defender

- 3.1 Attacker Pivot: From TTS Resource Theft to 300M Req/Day Pure EDoS
- 3.2 The Cloud Armor Trap: USD 1/Day Origin vs. USD 225/Day WAF Fees
- 3.3 Midnight Emergency GCLB Teardown & Flat-Rate WAF Evaluation

The Attacker's Pivot: From Resource Theft to Pure EDoS

Strategic Shift in Attack Vector

- **Phase 1 Blocked (`POST /synthesize`)** SHA-256 PoW made unauthorized TTS generation computationally infeasible.
- **Phase 2 Retaliation (`GET /challenge` & `/`)** Botnet pivoted to unauthenticated endpoints that require **zero PoW**.
- **300 Million Requests / Day** A **100x volumetric surge** (~3,500–10,000 RPS) designed purely to inflate cloud request & WAF bills.



The Cloud Armor Trap: 99.9% Blocked, \$6,750/mo Bill

99.9%

Cloud Armor Block Rate

Origin CPU stayed calm & 100% healthy

\$1/day

Cloud Run Origin Cost

Container compute completely protected

\$225/d

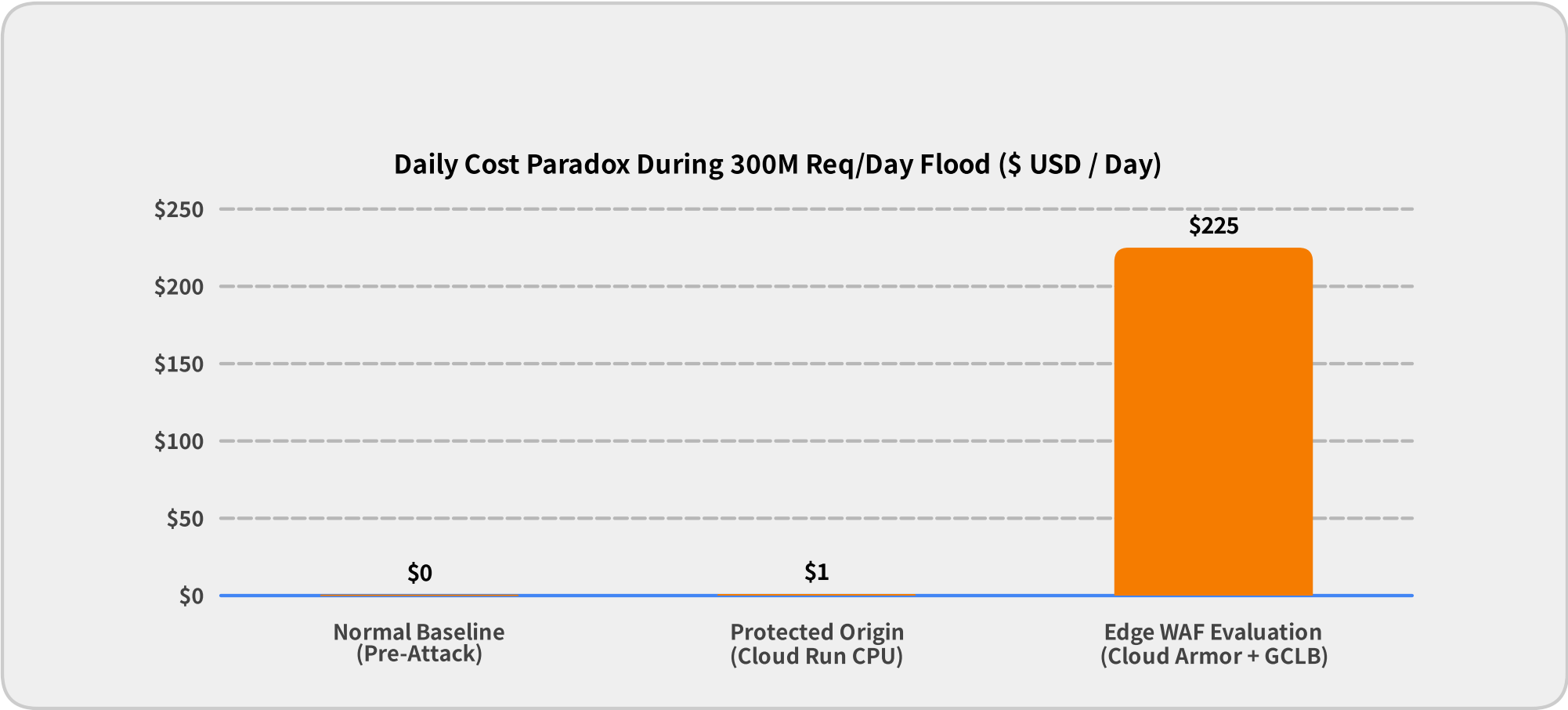
Cloud Armor + GCLB Fee

\$0.75 per 1M evaluated requests at edge

\$6,750

Projected Monthly Bill

675x over our \$10/mo hobby budget!

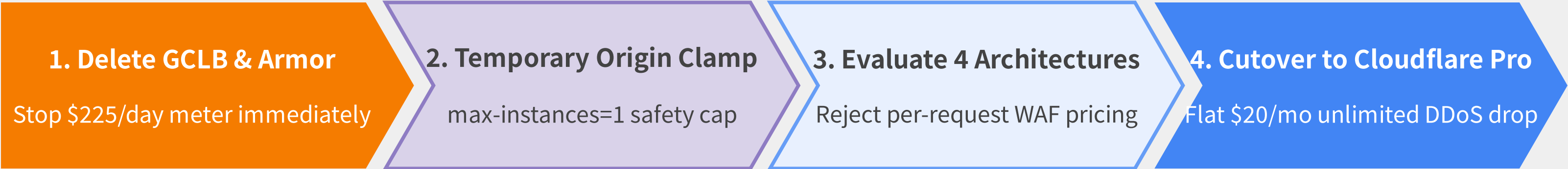


Why Blocking Traffic at the Edge Still Cost \$225/Day

Billing Component	Unit Pricing Model	300M Req/Day Volume	Daily Cost
Cloud Armor Standard	\$0.75 per 1M requests evaluated	300M blocked at edge	\$225.00 / day
GCLB Forwarding & L7	Hourly rule + L7 processing	300M TLS terminations	+\$18.00 / day

Midnight Emergency: Tearing Down GCLB & Choosing a Flat-Rate WAF

Midnight Emergency Cutover Sequence (Stopping the \$225/Day Bleeding)



Architectural Comparison: Pay-Per-Request Cloud WAFs vs. Flat-Rate Edge Proxy

Defense Option	Base Fee	Per-Request WAF Fee	Cost @ 300M Req/Day	Verdict
Google Cloud Armor (Standard)	\$5 / mo + GCLB	\$0.75 / 1M evaluated	~\$6,750 / mo	REJECT: EDoS Bankruptcy
AWS WAF / Azure WAF	\$5–10 / mo + ALB	\$0.60 / 1M evaluated	~\$5,400 / mo	REJECT: Per-Req Trap
Self-Hosted Nginx VPS	\$10 / mo fixed	\$0 (Fixed Bandwidth)	\$10 / mo	REJECT: 1 NIC Saturates
Cloudflare Pro (Unmetered L7)	\$20 / mo FLAT	\$0.00 (Unmetered Drop)	\$20 / mo FLAT	SELECTED: Zero EDoS Risk

ACT / CHAPTER

04



Act 4 of 4

Multi-Layered Edge Victory

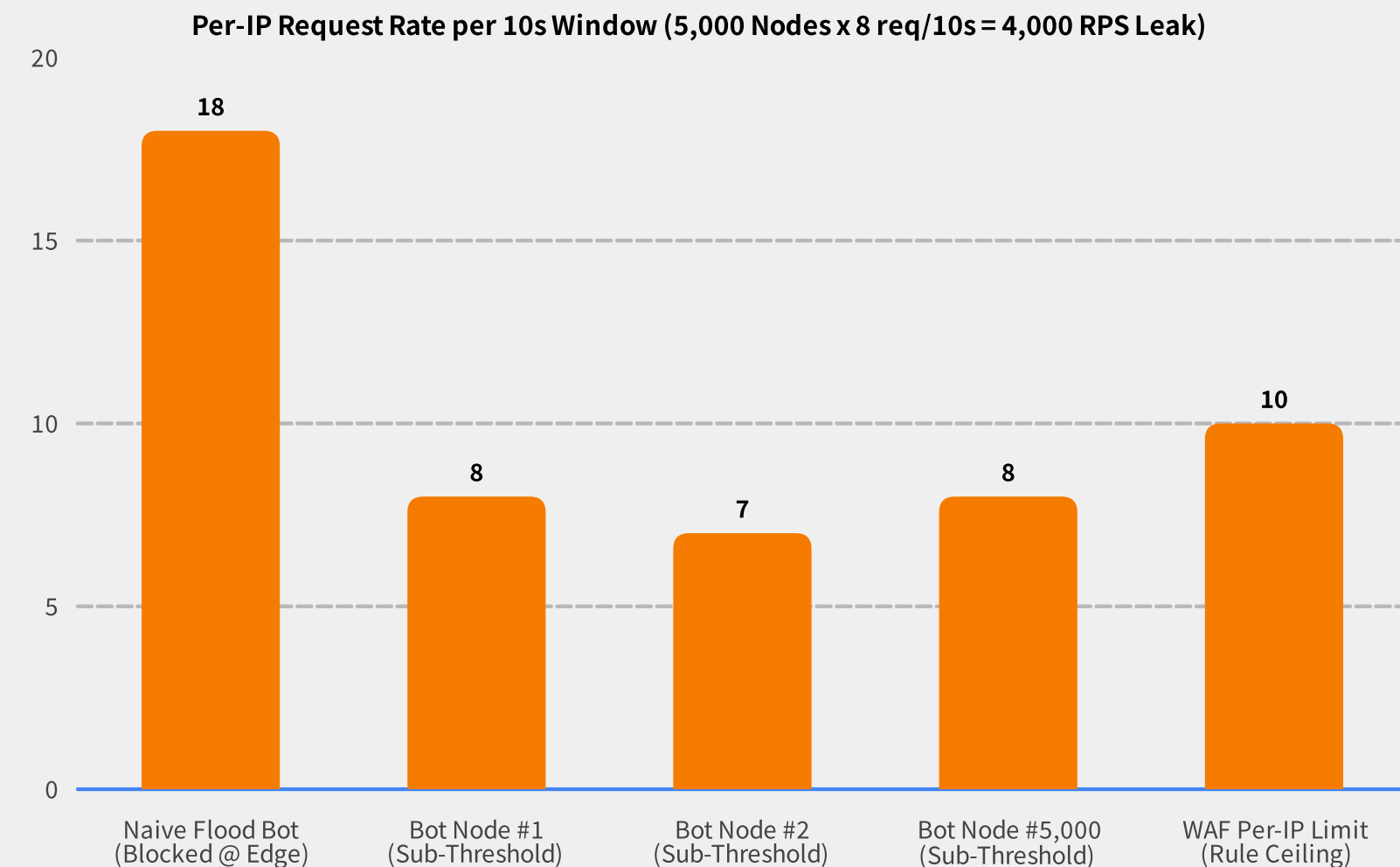
Traffic Forensics, 5 Defense Pillars, and AI Adversary Profiling

- 4.1 Cloudflare Pro Migration & The Sub-Threshold Botnet Leak (25%)
- 4.2 Traffic Forensics: Datacenter VMs vs. Hijacked Residential PCs
- 4.3 The 5-Layer Defense Architecture, AI Botnet Profile & SRE Rules

Cloudflare Pro Migration & The 25% Sub-Threshold Leak

Why Per-IP Rate Limits Leaked

- **Flat \$20/mo Cost Secured** Cut over DNS to Cloudflare Pro; 75% of the volumetric flood was blocked immediately at \$0 overage.
- **10 Req / 10s Per-IP Threshold** Setting stricter per-IP limits risked blocking legitimate Japanese offices & carrier NATs.
- **Distributed Sub-Threshold Swarm 5,000+ bot IPs** throttled down to **6–8 req / 10s each**, leaking **4,000 RPS (25%)** to origin!

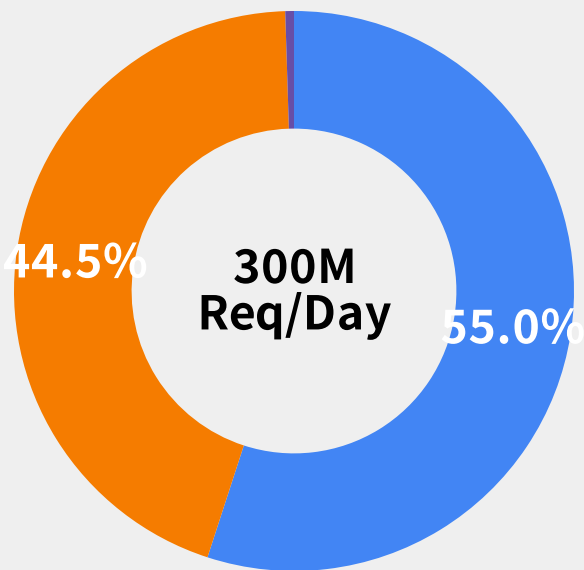


Math of Sub-Threshold Evasion: 5,000 IPs x 0.8 RPS/IP = 4,000 RPS hitting Cloud Run without triggering IP limits!

Traffic Forensics: Decomposing the Two Botnet Fleets

- **Log Forensics Breakthrough:** Inspecting Cloudflare request logs by **ASN**, **HTTP Protocol Version**, and **User-Agent** revealed the attack was actually **two distinct botnet fleets**.
- **Protocol Mismatch Tell:** Fleet B spoofed modern Chrome/Safari User-Agents (**Chrome/134**), but its proxy software spoke **HTTP/1.1** (real browsers always negotiate **HTTP/2 or HTTP/3!**).

Attack Fleet Breakdown



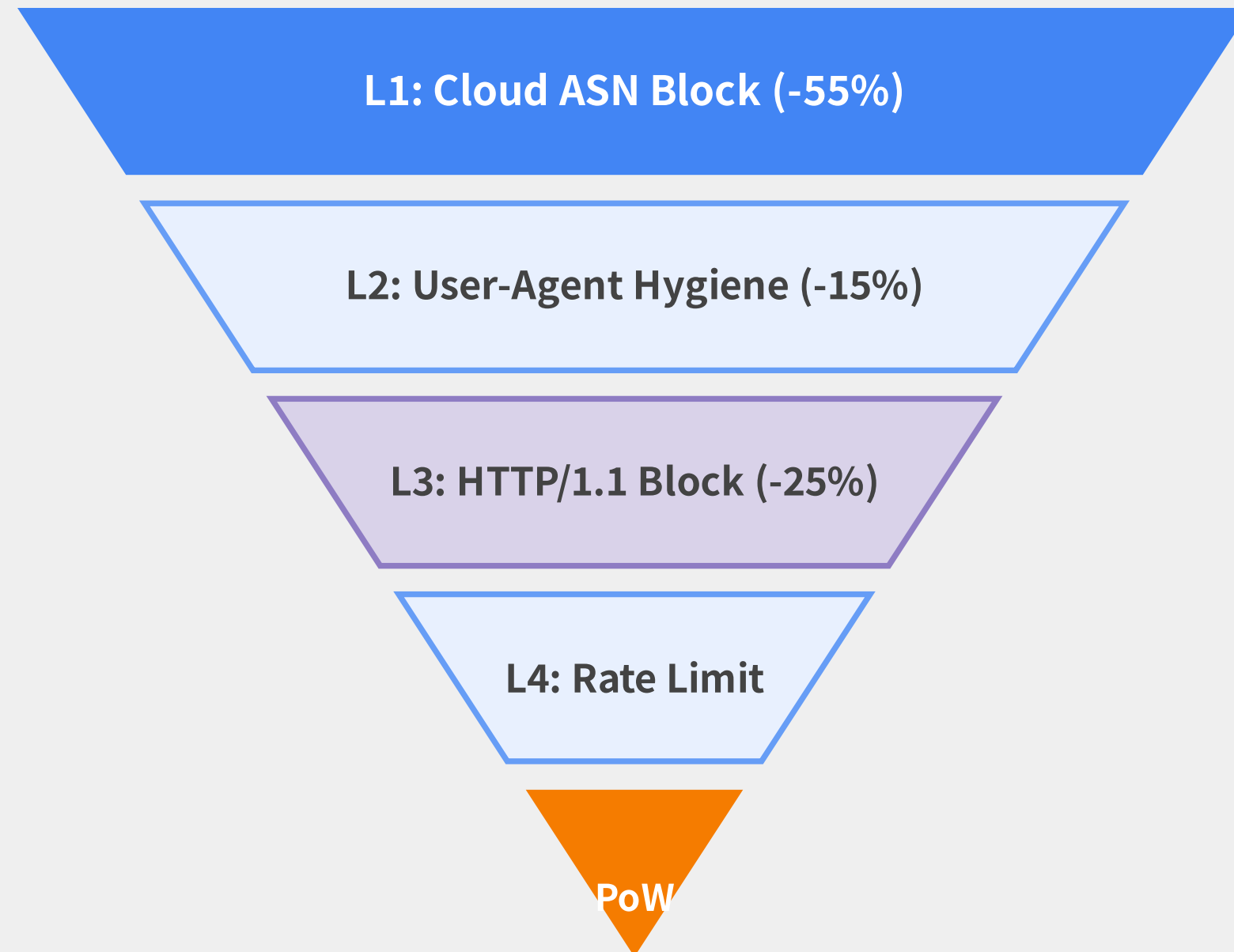
- Fleet A: Cloud ASNs (55%)
- Fleet B: Residential HTTP/1.1 (44.5%)
- Real Users: HTTP/2+3 (0.5%)

Forensic Fingerprint Comparison: Fleet A vs. Fleet B vs. Real Users

Signal	Fleet A: Cloud VMs (55%)	Fleet B: Residential (44.5%)	Real Humans (0.5%)
Network ASN	Hosting (AWS, GCP, OVH)	JP Home Fiber ISPs	JP Consumer ISPs
Rate per IP	High Burst (20–100 / 10s)	Stealth (6–8 req / 10s)	1–3 req / min
User-Agent	python / Go / Old Chrome	Spooled Chrome 130+	Modern Chrome / Safari
HTTP Version	HTTP/1.1 or HTTP/2	HTTP/1.1 ONLY (Tell!)	HTTP/2 or HTTP/3
WAF Action	Block Hosting ASNs	Block HTTP/1.1 @ Edge	Zero-Friction Pass

The 5-Layer Defense-in-Depth Architecture

5 Orthogonal Defense Layers: Filtering 300M Req/Day Down to 100% Clean Origin Traffic



L1: Network ASN Filter (Cloudflare WAF)

Drop all hosting/VPS ASNs (AWS, GCP, Azure, OVH, Hetzner)



L2: User-Agent Sanity Check (Cloudflare WAF)

Drop empty UAs, curl/python/go clients, and Chrome < v110



L3: Wire Protocol Enforcement (ALPN Check)

Block HTTP/1.1 requests; require HTTP/2 or HTTP/3 (QUIC)



L4: Tighten Edge Rate Limit (Flat-Rate Edge)

Composite IP + TLS fingerprint limit on GET /challenge



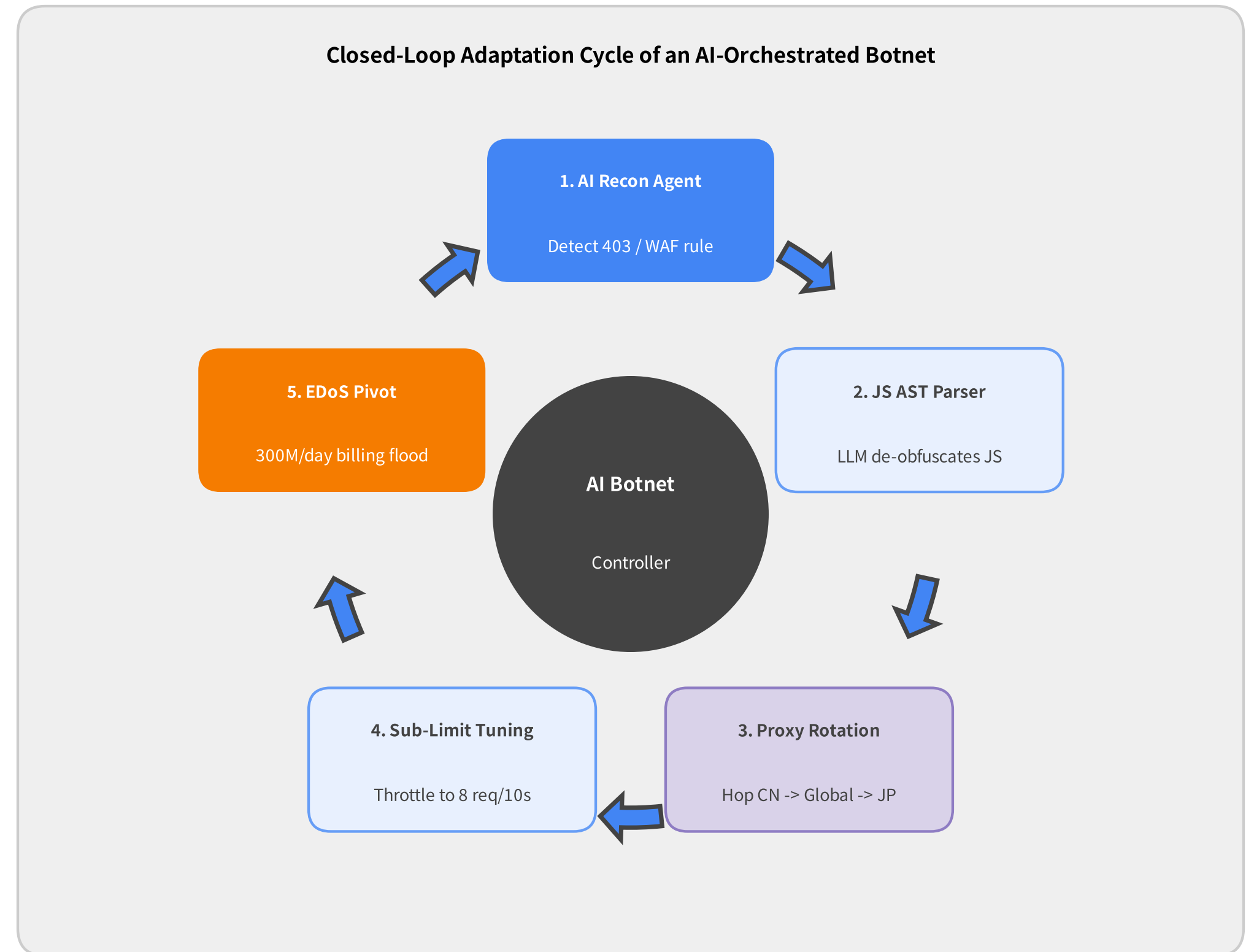
L5: Asymmetric SHA-256 Proof-of-Work (App Layer)

Signed seed + Web Worker nonce (<50ms) for synthesis API

Threat Actor Profile: Anatomy of an AI-Assisted Adversary

Behavioral Forensics

- **LLM-Speed Adaptation (<3 Hours)** De-obfuscated our custom frontend JS bundle and replicated HMAC signing in under 180 minutes.
- **Automated Threshold Probing** Tuned 5,000+ nodes to 6–8 req/10s to stay just beneath our 10 req/10s WAF rule.
- **Persistent Multi-Week Testing** Used our live production endpoint as a continuous adversarial training sandbox.



Key Takeaways: 4 Golden Rules for Serverless DDoS Defense

4 SRE Golden Rules for Serverless & Edge Security

Rule 1: Never Expose Serverless Direct

Lock down *.run.app to internal/LB only.
Always place a flat-rate Anycast Edge
in front of auto-scaling compute.

Rule 2: Audit WAF EDoS Pricing

Pay-per-request WAFs (\$0.75/M) can
bankrupt you during a 300M/day flood
even when blocking 99.9% of traffic.

Rule 3: Layer Orthogonal Signals

Single rules fail. Combine ASN blocks,
UA hygiene, HTTP/2+ ALPN checks,
and composite edge rate limits.

Rule 4: Use Asymmetric Proof-of-Work

Client JS secrets get reverse-engineered
in hours. Use SHA-256 Web Worker PoW
to make CPU abuse cost the attacker.

